PRACTICAL NO 07

AIM: Write a program to find the inorder, preorder and postorder traversal of a binary tree.

THEORY:

WRITE DOWN ABOUT TREE TRAVERSAL (PREORDER, INORDER, POSTORDER)

ALGORITHM: Algorithm Inorder(node):

1. If node is NULL:

return

2. Call Inorder(node->left) // visit left subtree

3. Visit (print) node->data // process root

4. Call Inorder(node->right) // visit right subtree

Algorithm Preorder(node):

1. If node is NULL:

return

2. Visit (print) node->data // process root

3. Call Preorder(node->left) // visit left subtree

4. Call Preorder(node->right) // visit right subtree

PROGRAM: Algorithm Postorder(node):

1. If node is NULL:

return

2. Call Postorder(node->left) // visit left subtree

3. Call Postorder(node->right) // visit right subtree

4. Visit (print) node->data // process root

1. Create root node.

2. Insert left and right child nodes to form a tree.

3. Print "Inorder traversal:" and call Inorder(root).

4. Print "Preorder traversal:" and call Preorder(root).

5. Print "Postorder traversal:" and call Postorder(root).

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Inorder Traversal (L -> Root -> R)
void inorder(struct Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

// Preorder Traversal (Root -> L -> R)
void preorder(struct Node* root) {
```

```
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

// Postorder Traversal (L -> R -> Root)
void postorder(struct Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

// Main Function
int main() {
    // Creating a simple binary tree
    /*
          1
         / \
        2   3
       / \
      4   5
    */
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    printf("Inorder traversal: ");
```

```
    inorder(root);
    printf("\n");

    printf("Preorder traversal: ");
    preorder(root);
    printf("\n");

    printf("Postorder traversal: ");
    postorder(root);
    printf("\n");

    return 0;
}
```

OUTPUT

Inorder traversal: 4 2 5 1 3

Preorder traversal: 1 2 4 5 3

Postorder traversal: 4 5 2 3 1